OutreachIQ: An AI Agent System for Grounded B2B Sales Outreach

Project Report

1. Abstract

OutreachIQ is a working product that automates the hardest, most repetitive part of B2B sales: finding the right companies to sell to and writing a first email worth reading. A user pastes the URL of their own company. The system reads that website, works out what the company sells and who its ideal customer is, then goes out to the live web to find real companies that match. For each match it does its own research, scores how good a fit it is, maps the prospect's likely problems to proof the seller actually has, and drafts a personalized outreach email. The emails can be reviewed and sent straight from the user's own Gmail account.

The project is built on Microsoft Azure AI Foundry for its reasoning and grounding layer, and it deliberately mixes two different model providers so that each task runs on the model that fits it best. The whole thing runs end to end today, not as a mockup.

2. Problem Statement

Sales teams waste an enormous amount of time on work that feels like research but is really just manual labor. A rep has to guess which companies might need the product, open each website one by one, read enough to form an opinion, figure out a reason to reach out, and then write an email that does not sound like a template. Most of this gets skipped under pressure, which is why so much cold outreach is generic and ignored.

The common "AI email writer" tools do not actually solve this. They write faster, but they write from nothing. They do not know who to contact, they do not research the target, and they have no idea whether the product is even a good fit. The result is faster spam.

OutreachIQ was built to close that gap. The goal was a system that starts from a single input, the seller's own website, and produces a short list of genuinely relevant prospects, each with research behind it and an email grounded in facts about both sides.

3. What the Product Does

The product follows one clean flow from the user's point of view.

Step one, understand the seller. The user pastes their company URL. OutreachIQ fetches and reads the site, including pages like the about and pricing pages, and extracts a structured profile: what the company sells, its value propositions, and an ideal customer profile covering industry, company size, stage, geography, buying signals, and anti-signals. The user can edit any of this before continuing, because the person always knows their business better than a model does.

Step two, optional context. The user can upload their own documents, such as case studies, sales decks, or product one-pagers. These go into a knowledge base so that later emails can cite real proof the website never shows.

Step three, connect sending. The user enters the name they want emails signed with, and connects their Gmail account once. Nothing is sent without review.

Step four, run. The user picks how many prospects to find and whether to search locally, internationally, or both, and starts a run. From here the system takes over. It searches the live web for companies that match the ideal customer profile, then runs each discovered company through a five stage agent pipeline. Progress is shown live, one row per company, with a stepper that fills in as each stage completes.

Step five, review and send. When the run finishes, the user sees each prospect with the full research, the fit score and reasoning, the mapped pain points, the chosen hook, and a ready to send email. The user reviews, adjusts the recipient address, and sends from their own Gmail.

A dashboard ties it together with the prospects found, emails ready, average fit, and a breakdown of which model provider handled which share of the work.

---

4. System Architecture

OutreachIQ is a single product split into a backend service and a web frontend.

The **backend** is a Python service built with FastAPI. It exposes a REST API for the frontend and uses server sent events to stream live run progress. Data is stored in SQLite through SQLAlchemy. Each run executes on its own background worker thread so the API stays responsive while agents work.

The **frontend** is a React application built with Vite and styled with Tailwind. It is a single page product with a premium, monochrome look. The main screens are the setup page where the company is analyzed, the new run page, the live run view, the prospects review page, and the dashboard.

Between them sits the part that makes the product interesting: the agent and model layer.

---

5. The Agent Pipeline

Every discovered company is run through five agents in order. Each agent has one job and passes its output to the next.

1. **Researcher.** Turns a raw company blurb into structured facts: industry, size estimate, what they do, and growth or buying signals. It is told to use only the text it is given and to mark anything absent as unknown, so it does not invent details.

1. **Fit Scorer.** Scores the prospect from 0 to 100 against the seller's own ideal customer profile and justifies the score in one sentence. This agent reads from the knowledge base, so the score is grounded in the seller's real positioning rather than a guess.

1. **Pain Mapper.** Identifies the top two problems this prospect likely has that the seller actually solves, and names the single most relevant case study as proof. This is also grounded in the knowledge base, which is what keeps it honest.
1. **Hook Finder.** Picks one short personalization angle, no more than a dozen words, that gives the email a genuine reason to exist.
1. **Email Drafter.** Writes the outreach email in the seller's voice, using the hook and the mapped pains, citing the proof point naturally, and signing off with the user's real name and company. It is explicitly told never to leave placeholders like "[Your Name]". This agent is grounded in the knowledge base too, so the email matches the tone of the seller's own winning emails.

The discovery step that feeds this pipeline is handled separately by a prospector that searches the live web, scores how well each result matches, and hands the top companies to the pipeline.

---

6. Where Azure AI Foundry Is Used

Azure AI Foundry is the core of the system, not an add on. It provides three things the product depends on.

Reasoning. The judgment heavy agents, fit scoring, pain mapping, and email drafting, run on an Azure deployed chat model (gpt-4.1-mini). These are the steps where quality matters most, because a bad score or a hollow email wastes the whole run.

Grounding through a knowledge base. The seller's profile, uploaded documents, and sample emails live in an Azure vector store. The grounded agents call the Azure Responses API with its file search tool, which means their answers are built from the seller's real material and come back with citations pointing to the source. This is what separates OutreachIQ from a tool that just makes things up. When the email claims a proof point, that proof point is traceable.

Embeddings. Documents are embedded with Azure's text-embedding-3-small model when they go into the knowledge base, so file search can find the relevant passages.

All of this runs against the Foundry account endpoint using the preview API surface that exposes the Responses API, vector stores, and chat in one place.

---

7. Multi-Model Orchestration

A single model for every task is either too expensive or too weak. OutreachIQ runs a router that sends each job to the right provider.

Azure Foundry handles judgment and grounded work. Fit scoring, pain mapping, and email writing all go through Azure, because these need careful reasoning and access to the knowledge base.

A fast, cheap provider handles bulk work. The researcher, the hook finder, query generation, and company extraction during discovery run on a Groq hosted open model. These jobs are high

volume and more mechanical, so paying premium rates for them would be wasteful.

Every model call, from either provider, records its token usage and estimated cost into a per run ledger. The dashboard then shows the split between the two providers. This is the orchestration thesis made visible: the system proves it is routing work deliberately, and it can account for exactly what each run cost and where.

---

8. Live Web Discovery

The discovery layer is what makes the prospects real rather than a fixed list.

When a run starts, the system generates search queries from the seller's ideal customer profile. It favors queries that surface lists of companies in a given industry and location rather than vague marketing phrases. It respects the geography the user chose, local, international, or both, using country detection so a seller in India searching locally does not get flooded with results from elsewhere.

For search it prefers Serper, a fast Google results API, and falls back to an Apify Google scraper if needed. This choice mattered a great deal in practice. An earlier approach took over two and a half minutes per run and sometimes returned noise; the current one returns real, relevant companies in under twenty seconds.

Once a company is found, its website has to be read, and modern sites are mostly JavaScript applications that a plain fetch cannot see. OutreachIQ renders each page in a real headless Chrome browser locally, so single page applications load fully before their text is extracted. This brought scraping down to roughly seven to eleven seconds per site while actually working on sites that the simple approach could not read at all.

---

9. Email Sending

Drafting an email is only half the job. OutreachIQ can send it from the user's own Gmail account.

Sending uses Google OAuth. The user connects Gmail once and grants only the permission to send mail. The system never sees or stores a password. It holds only a refresh token, which is the standard, safe way to do this. Emails are composed and sent through the Gmail API so they come from the seller's real address and land in the prospect's inbox as a genuine message.

The design is review then send, not blind auto send. Every email is a draft until the user looks at it and chooses to send. This was a deliberate decision. It respects anti spam law and data protection rules, and it keeps a human in control of anything that goes out under their name.

---

10. Technology Stack

Backend: Python, FastAPI, SQLAlchemy with SQLite, server sent events for streaming.

Frontend: React, Vite, Tailwind, React Router, with a floating dock navigation and live animated run progress.

AI and grounding: Microsoft Azure AI Foundry (chat model gpt-4.1-mini, embeddings via text-embedding-3-small, vector store knowledge base, Responses API with file search for grounded answers and citations).

Bulk model provider: Groq hosted open model for high volume extraction and research.

Web discovery: Serper for fast search results, with an Apify fallback.

Scraping: Playwright driving a local headless Chrome to render JavaScript sites.

Email: Google OAuth and the Gmail API.

---

11. Results

The product works end to end. From a single pasted URL it produces analyzed company profiles, a list of real matching companies found live on the web, full research and fit scoring for each, and personalized, grounded emails that can be sent from the user's own account.

Performance was a real focus. Company analysis settled at around twenty five seconds, discovery dropped from over one hundred and fifty seconds to under twenty, and a full run for a couple of prospects came down from over three minutes to under a minute, with each prospect's five stage pipeline taking roughly sixteen seconds. The cost ledger confirms that the expensive Azure model is used only where judgment and grounding are needed, while bulk work runs on the cheaper provider.

---

12. Challenges and How They Were Solved

Several problems came up during the build and each pushed the design somewhere better.

Modern websites would not scrape with a simple request because their content loads through JavaScript. The fix was to render pages in a real browser, which made scraping both reliable and reasonably fast.

Web discovery was slow and sometimes returned irrelevant results from the first search approaches tried. Switching to a proper search results API fixed both speed and quality.

The Gmail connection initially failed with a code verifier mismatch, a known issue with the OAuth security exchange when the two halves of the flow do not share state. Persisting that value between the two steps resolved it, and Gmail now connects cleanly.

A few user interface bugs showed up after clearing old data, where a stale run reference left a page loading forever. These were fixed by detecting the missing run and falling back to a sensible empty state.

---

13. Future Work

The per prospect pipeline currently runs one company at a time. Running several in parallel would cut total run time further, though it needs care around database and rate limit concurrency. Beyond that, natural extensions include reply tracking, follow up sequences, a shared knowledge base across a team, and support for sending providers other than Gmail.

---

14. Conclusion

OutreachIQ shows that useful B2B outreach can be automated without giving up quality or honesty. By building on Azure AI Foundry for grounded reasoning, routing work across two model providers so each task runs where it belongs, discovering real companies on the live web, and keeping a human in control of anything that gets sent, the system produces outreach that is researched, personalized, and traceable to real facts. It is a complete product rather than a demo, and every part of the flow, from a single pasted URL to a sent email, works today.